

Modüler Programlama

- Bu haftanın temel amacı, öğrencinin “tek dosyalı küçük C programı” mantığından çıkıp, daha düzenli ve profesyonel kod yapısına geçmesini sağlamaktır. Şimdiye kadar yazılan programlar çoğunlukla tek .c dosyasında, birkaç fonksiyonla ilerliyordu. Ancak gerçek uygulamalarda kod büyüdükçe her şeyi tek dosyada tutmak hem okumayı hem hata ayıklamayı hem de geliştirmeyi zorlaştırır.

Modüler Programlama

Preprocessor nedir?

- C dilinde yazdığımız kod, doğrudan derleyiciye gitmez. Önce preprocessor adı verilen bir ön işlem aşamasından geçer. Preprocessor, derlemeden önce çalışan bir mekanizmadır ve özellikle # ile başlayan satırları işler.

Örneğin:

- #include #define #ifndef #endif
gibi satırlar, derleyiciden önce değerlendirilir..

Modüler Programlama

Preprocessor'ın yaptığı iş, programı çalıştırmak değildir. O, kaynak kod üzerinde metinsel düzenlemeler yapar:

- başka dosyaları ekler,
- bazı isimleri başka ifadelerle değiştirir,
- belirli koşullara göre bazı satırları dahil eder veya etmez.

Bu nedenle anlamamız gereken ilk şey :

- Preprocessor, derlemeden önce çalışan bir hazırlık aşamasıdır.

Modüler Programlama

Çalışma Sırası:

- Kaynak kod yazılır (.c dosyası)
- Preprocessor direktifleri işlenir (# ile başlayan satırlar)
- Ortaya çıkan temiz kod derleyiciye gider
- Derleyici makine kodunu üretir

Modüler Programlama

- #include nedir?
- #include, başka bir dosyanın içeriğini bulunduğu yere ekleyen bir preprocessor komutudur. En basit mantıkla düşünersek, #include satırı şu işi yapar:
 - “Git, o dosyanın içeriğini buraya kopyala-yapıştır.
 - ”Örneğin: #include <stdio.h> dediğimizde, stdio.h dosyasındaki gerekli bildirimler programa eklenir.

Modüler Programlama

Benzer şekilde:

#include "ogrenci.h"

- dediğimizde kendi yazdığımız başlık dosyasının içeriği o noktaya eklenmiş olur.
- Burada önemli olan şey, #include'un dosyayı “çağırması” değil, metinsel olarak eklemesidir.

Modüler Programlama

#include <...> ile #include "... " farkı

C'de #include iki biçimde kullanılır:

#include <stdio.h>

#include "ogrenci.h"

Aradaki fark:

- <...> genellikle **standart kütüphane** başlık dosyaları için kullanılır.
- "... " ise genellikle **bizim yazdığımız dosyalar** için kullanılır. Derleyici bu iki biçimde dosyayı farklı yerlerde arar:
- <stdio.h> için sistemin standart include yollarına bakar.
- "ogrenci.h" için önce mevcut klasöre, sonra diğer include yollarına bakar.

Bu ayrım, öğrencinin kendi modüllerini yazarken çok önemlidir.

```
#include <stdio.h>           // Sistem kütüphaneleri için (sistem dizinlerinde arar)
#include "hesapla.h"         // Kullanıcı header'ları için (önce mevcut dizinde arar)
```

Modüler Programlama

Örnek:

```
// hesapla.h dosyası
```

```
int topla(int a, int b);
```

```
int carp(int a, int b);
```

```
// main.c dosyası
```

```
#include "hesapla.h"
```

```
int main() {
```

```
    int sonuc = topla(5, 3);
```

```
    return 0;
```

```
}
```

Preprocessor'dan sonra main.c şöyle görünür:

```
// main.c (preprocessor sonrası)
```

```
int topla(int a, int b);
```

```
int carp(int a, int b);
```

```
int main() {
```

```
    int sonuc = topla(5, 3);
```

```
    return 0;
```

```
}
```

Modüler Programlama

- `#define`, bir isim ile bir değer veya ifade arasında metinsel eşleştirme yapan preprocessor komutudur.
- Örneğin:
`#define PI 3.14`
`#define BOYUT 100`
- Burada `PI` veya `BOYUT` geçen yerler, derleme öncesinde ilgili değerlerle değiştirilir.
- `int dizi[BOYUT];` preprocessor sonrasında şöyle düşünülür: `int dizi[100];`
- `#define` değişken değildir. Bellekte yer ayırmaz. Sadece metin değiştirme yapar.
- `#define` bellekte yer ayırmaz,
- fonksiyon çağırısı yapmaz,
- sadece metinsel değiştirme yapar

```
#define PI 3.14159
#define MAX_BOYUT 100
#define MESAJ "Hosgeldiniz"
```

```
// Kullanım
float alan = PI * r * r;
int dizi[MAX_BOYUT];
printf("%s\n", MESAJ);
```

Modüler Programlama

#define ile makro kavramına kısa giriş

#define sadece sabit sayı tanımlamak için değil, bazen parametrelili ifadeler için de kullanılabilir.

Örneğin:

```
#define KARE(x) ((x) * (x))
```

Bu yapı makro olarak adlandırılır. Ancak başlangıç seviyesinde makrolar bazen kafa karıştırıcı olabilir çünkü preprocessor düzeyinde çalışırlar ve fonksiyon gibi davranıyor görünseler de gerçek fonksiyon değildirler.

```
// Yanlış:  
#define KARE(x) x * x  
int sonuc = KARE(3 + 2); // 3 + 2 * 3 + 2 = 11 (YANLIŞ!)  
  
// Doğru:  
#define KARE(x) ((x) * (x))  
int sonuc = KARE(3 + 2); // ((3 + 2) * (3 + 2)) = 25 (DOĞRU!)
```

Modüler Programlama

#ifndef / #define / #endif nedir?

- Bu üçlü yapı, özellikle header guard yapmak için kullanılır. Ama önce temel anlamı anlaşılmalıdır.
- #ifndef → “Eğer bu isim tanımlı değilse”
- #define → “Bu ismi tanımla”
- #endif → “Koşullu bölümü bitir”
- Örnek:

```
#ifndef OGR_H
#define OGR_H
/* içerik */
#endif
```

Bu yapı şunu söyler:

- Eğer OGR_H daha önce tanımlanmadıysa, içeriği ekle
- sonra OGR_H ismini tanımla
- bir daha aynı dosya gelirse tekrar ekleme

Bu mekanizma, aynı header dosyasının birden fazla kez işlenmesini önler.

```
hesapla.h
#define HESAPLA_H
#define HESAPLA_H

topla(int a, int b);
carp(int a, int b);

if
```

Modüler Programlama

- Modüler programlama, bir programı tek parça hâlinde yazmak yerine, mantıksal bölümlere ayırarak geliştirme yaklaşımıdır.

Örneğin bir öğrenci sistemi düşünelim:

- öğrenci bilgileriyle ilgili fonksiyonlar bir yerde
- not hesaplama fonksiyonları başka yerde
- ana program başka yerde olabilir

Bu sayede kod:

- daha düzenli olur
- daha okunur olur
- bakım yapmak kolaylaşır
- ekip çalışmasına uygun hâle gelir

Takım Çalışması

Ahmet: kullanıcı arayüzü (ui.c)

Mehmet: veritabanı işlemleri (db.c)

Ayşe: hesaplama fonksiyonları (hesap.c)

Aynı anda çalışabilirler

Modüler Programlama

Bir modül, programın belirli bir görevini üstlenen bağımsız parçadır. C dilinde modüller genelde şu dosya yapısıyla kurulur:

- ogrenci.h
- ogrenci.c
- main.c

Burada:

- ogrenci.h → dışarıya görün
- c → gerçek kod
- main.c → programın akışı

Bu yapı, öğrencinin “programı katmanlı düşünmesini” sağlar.

```
e/  
  
hesapla.h      (Header: Arayüz/Interface)  
hesapla.c      (Implementation: Gerçekleme)  
geometri.h  
geometri.c  
main.c        (Ana program)
```

Modüler Programlama

.c dosyasının rolü.

- c dosyası, fonksiyonların gerçek yazıldığı yerdir. Yani işin “uygulama” kısmı burada olur. Örnek:
- void ogrenciYazdir(...) {

...

}

Bunlar .c dosyasında bulunur.

.c dosyası:

- kodun gerçek gövdesini içerir
- derlenir
- nesne dosyasına (.o / .obj) dönüşür

Modüler Programlama

- .h dosyasının rolü
 - .h dosyası genelde:
- fonksiyon prototiplerini
- struct tanımlarını
- typedef'leri
- enum'ları
- sabit tanımları içerir.

Yani .h dosyası, modülün dış dünyaya sunduğu arayüzdür. Örnek:

```
struct Ogrenci { ... };
```

```
void ogrenciYazdir(struct Ogrenci o);
```

- Bu bilgiler main.c tarafından bilinmelidir ama fonksiyonun iç gövdesi bilinmek zorunda değildir.

Modüler Programlama

Arayüz ile gerçekleştirme ayrımı

- Arayüz (interface): dışarıya ne sunuyorsun?
- Gerçekleme (implementation): bunu içeride nasıl yapıyorsun?

Örneğin:

- ogrenci.h dosyası “hangi fonksiyonlar var?” sorusuna cevap verir.
- ogrenci.c dosyası “bu fonksiyonlar nasıl çalışıyor?” sorusuna cevap verir.

Modüler Programlama

Basit bir öğrenci modülü:

ogrenci.h

```
#ifndef OGRENCI_H
#define OGRENCI_H
struct Ogresnci {
int no;
char ad[20];
};
void yazdir(struct Ogresnci o);

#endif
```

ogrenci.c

```
#include <stdio.h>
#include "ogrenci.h"

void yazdir(struct Ogresnci o){
printf("%d %s\n", o.no, o.ad);
}
```

main.c

```
#include "ogrenci.h"

int main(void){
struct Ogresnci o = {101, "Ali"};
yazdir(o);
return 0;
}
```

Modüler Programlama

Derleme nedir?

- Derleme, yazdığımız C kodunun makinenin anlayacağı biçime çevrilmesi sürecidir. Ancak bu süreç tek aşamalı değil, birkaç aşamalıdır.

Basitçe:

- önce preprocessor çalışır,
- sonra derleyici kodu çevirir,
- sonra assembler/makine düzeyine iner,
- en sonunda linker tüm parçaları birleştirir.
- Başlangıç seviyesinde en önemli kısım şudur: Her .c dosyası ayrı ayrı derlenebilir ve sonra birleştirilebilir.

Modüler Programlama

Tek dosya derleme

Tek dosyalı bir program için derleme mantığı basittir:

gcc main.c -o program

Burada:

- main.c kaynak dosyadır
- -o program çıktı dosyasının adını belirtir

Bu yapı küçük programlar için yeterlidir.

Çok dosya derleme

Program birden fazla .c dosyasına bölündüğünde derleme de genişler.

Örnek:

gcc main.c ogrenci.c -o program

Burada hem main.c hem ogrenci.c derlenir ve sonra tek çalıştırılabilir dosyada birleştirilir.

Modüler Programlama

Derleyici neden .h dosyasını ayrıca derlemiyor?

Çünkü .h dosyaları genellikle tek başına çalıştırılabilir kod içermez. Onlar, .c dosyalarının içine #include ile eklenir.

Yani:

- .c dosyaları derlenir
- .h dosyaları preprocessor tarafından metin olarak eklenir

Modüler Programlama

- **bildirim (declaration):** derleyiciye “böyle bir şey var” demek
- **tanım (definition):** onun gerçek içeriğini vermek

Örnek:

void yazdir(struct Ogrenci o); // bildirim

void yazdir(struct Ogrenci o){ // tanım

...

}

.h dosyalarında genelde bildirimler bulunur.

.c dosyalarında tanımlar bulunur.

Modüler Programlama

Header guard neden gereklidir?

Bir header dosyası bazen dolaylı olarak birden fazla kez eklenebilir.

Örneğin:

- main.c içinde #include "ogrenci.h"
- ayrıca not.h içinde de #include "ogrenci.h"

Bu durumda ogrenci.h iki kez işlenebilir.

Eğer bu dosyanın içinde:

- struct tanımı
- typedef
- enum
- fonksiyon bildirimleri varsa, bazıları tekrar tekrar eklenir ve hata oluşabilir.

Modüler Programlama

Klasik header guard şablonu

Header guard'ın klasik yapısı şöyledir:

```
#ifndef OGRENCI_H
```

```
#define OGRENCI_H
```

```
/* içerik */
```

```
#endif
```

Bu yapı adım adım şöyle çalışır:

1. OGRENCI_H tanımlı mı diye bak
2. Tanımlı değilse içeriği ekle
3. Sonra OGRENCI_H ismini tanımla
4. Bir daha aynı dosya gelirse içeriği tekrar ekleme

Modüler Programlama

```
// ogrenci.h
#ifndef OGRENCI_H
#define OGRENCI_H

#define MAX_OGRENCI 100

struct Ogresnci {
    char ad[50];
    char soyad[50];
    int no;
    float not_ortalamasi;
};

void ogrenci_ekle(struct Ogresnci *o);
void ogrenci_goster(struct Ogresnci *o);
float ortalama_hesapla(float *notlar, int adet);

#endif // OGRENCI_H
```

İkinci dahil edilişte:

1. #ifndef OGRENCI_H → OGRENCI_H tanımlı mı?
Evet
2. Tüm blok atlanır
3. #endif → Hiçbir şey eklenmemiş olur

Modüler Programlama

```
proje/  
├─ hesapla.h  
├─ hesapla.c  
├─ geometri.h  
├─ geometri.c  
└─ main.c
```

hesapla.h:

```
#ifndef HESAPLA_H  
#define HESAPLA_H
```

```
int topla(int a, int b);  
int carp(int a, int b);
```

```
#endif
```

hesapla.c:

```
#include "hesapla.h"
```

```
int topla(int a, int b) {  
    return a + b;  
}
```

```
int carp(int a, int b) {  
    return a * b;  
}
```

Modüler Programlama

```
proje/  
├─ hesapla.h  
├─ hesapla.c  
├─ geometri.h  
├─ geometri.c  
└─ main.c
```

geometri.h:

```
#ifndef GEOMETRI_H  
#define GEOMETRI_H
```

```
float daire_alan(float r);  
float dikdortgen_alan(float en, float boy);
```

```
#endif
```

geometri.c:

```
#include "geometri.h"  
#define PI 3.14159
```

```
float daire_alan(float r) {  
    return PI * r * r;  
}
```

```
float dikdortgen_alan(float en, float boy) {  
    return en * boy;  
}
```

Modüler Programlama

```
proje/  
├─ hesapla.h  
├─ hesapla.c  
├─ geometri.h  
├─ geometri.c  
└─ main.c
```

main.c:

```
#include <stdio.h>
```

```
#include "hesapla.h"
```

```
#include "geometri.h"
```

```
int main() {
```

```
    int toplam = topla(5, 3);
```

```
    int carpim = carp(4, 6);
```

```
    float alan = daire_alan(5.0);
```

```
    printf("Toplam: %d\n", toplam);
```

```
    printf("Carpim: %d\n", carpim);
```

```
    printf("Daire alani: %.2f\n", alan);
```

```
    return 0;
```

```
}
```

Modüler Programlama

```
proje/  
├─ hesapla.h  
├─ hesapla.c  
├─ geometri.h  
├─ geometri.c  
└─ main.c
```

Derleme Adımları:

Yöntem 1: Hepsi birden

```
gcc main.c hesapla.c geometri.c -o program  
./program
```

Yöntem 2: Ayrı ayrı derleme (Önerilen)

Her .c dosyasını obje dosyasına (.o) derle

```
gcc -c hesapla.c -o hesapla.o
```

```
gcc -c geometri.c -o geometri.o
```

```
gcc -c main.c -o main.o
```

Obje dosyalarını birleştir (link)

```
gcc hesapla.o geometri.o main.o -o program
```

Çalıştır

```
./program
```

gcc: Derleyiciyi çağır.

-o :Derleme sonucunda oluşacak olan çalıştırılabilir (executable) dosyanın ismini belirlemenizi sağlar.

./program ifadesindeki ./ kısmı, işletim sistemine dosyanın nerede olduğunu söyler.

- **.(Nokta):** "Şu an içinde bulunduğum klasör" anlamına gelir.
- **/(Slash):** Klasör ayracıdır.

Avantajı: Sadece değişen dosya yeniden derlenir.

Modüler Programlama

Aşağıdaki klasör yapısına sahip, çok dosyalı bir C projesi geliştiriniz:

Aşağıdaki dosya yapısına sahip bir hesap makinesi programı yazınız:

matematik.h, matematik.c, giris.h, giris.c, main.c

- matematik modülü: topla, cikar, carp, bol
- giris modülü: kullanıcıdan sayı ve işlem alma
- main.c: program akışı ve sonucu ekrana yazdırma

Kurallar:

- Header guard kullanınız.
- Çok dosyalı yapı korunacaktır.
- Bölme işleminde sıfıra bölme kontrolü yapılacaktır.

```
hesap_makinesi/  
├─ matematik.h  
├─ matematik.c  
├─ giris.h  
├─ giris.c  
└─ main.c
```

Modüler Programlama

matematik.h:

```
#ifndef MATEMATIK_H  
#define MATEMATIK_H
```

```
// Fonksiyon prototipleri
```

```
int topla(int a, int b);
```

```
int cikar(int a, int b);
```

```
int carp(int a, int b);
```

```
float bol(int a, int b);
```

```
// Sabitler
```

```
#define HATA_KODU -999
```

```
#endif
```

```
hesap_makinesi/  
├─ matematik.h  
├─ matematik.c  
├─ giris.h  
├─ giris.c  
└─ main.c
```

Modüler Programlama

matematik.c:

```
#include "matematik.h"
```

```
int topla(int a, int b) {  
    return a + b;  
}
```

```
int cikar(int a, int b) {  
    return a - b;  
}
```

```
int carp(int a, int b) {  
    return a * b;  
}
```

```
float bol(int a, int b) {  
    if (b == 0) {  
        return HATA_KODU;  
    }  
    return (float)a / b;  
}
```

```
hesap_makinesi/  
├─ matematik.h  
├─ matematik.c  
├─ giris.h  
├─ giris.c  
└─ main.c
```

Modüler Programlama

Giris.h:

```
#ifndef GIRIS_H
#define GIRIS_H

// Kullanıcı girdisi alma fonksiyonları
int sayi_al(const char *mesaj);
char islem_al();

#endif
```

```
hesap_makinesi/
├─ matematik.h
├─ matematik.c
├─ giris.h
├─ giris.c
└─ main.c
```

Modüler Programlama

Giris.c:

```
#include "giris.h"  
#include <stdio.h>
```

```
int sayi_al(const char *mesaj) {  
    int sayi;  
    printf("%s", mesaj);  
    scanf("%d", &sayi);  
    return sayi;  
}
```

```
char islem_al() {  
    char islem;  
    printf("Islem secin (+, -, *, /): ");  
    scanf(" %c", &islem);  
    return islem;  
}
```

```
hesap_makinesi/  
├─ matematik.h  
├─ matematik.c  
├─ giris.h  
├─ giris.c  
└─ main.c
```

```

#include <stdio.h>
#include "matematik.h"
#include "giris.h"
int main() {
    int sayi1 = sayi_al("Birinci sayi: ");
    int sayi2 = sayi_al("Ikinci sayi: ");
    char islem = islem_al(); float sonuc;
    switch(islem) {
        case '+':
            sonuc = topla(sayi1, sayi2);
            break;
        case '-':
            sonuc = cikar(sayi1, sayi2);
            break;
        case '*':
            sonuc = carp(sayi1, sayi2);
            break;
        case '/':
            sonuc = bol(sayi1, sayi2);
            if (sonuc == HATA_KODU) {
                printf("Hata: Sifira bolme!\n");
                return 1;
            }
            break;
        default:
            printf("Gecersiz islem!\n");
            return 1;
    }
    printf("Sonuc: %.2f\n", sonuc);
    return 0;
}

```

Modüler Programlama

```

hesap_makinesi/
├── matematik.h
├── matematik.c
├── giris.h
├── giris.c
└── main.c

```

Main.c:

Modüler Programlama

derleme:

```
gcc -c matematik.c -o matematik.o
gcc -c giris.c -o giris.o
gcc -c main.c -o main.o
gcc matematik.o giris.o main.o -o hesap_makinesi
./hesap_makinesi
```

```
hesap_makinesi/
├─ matematik.h
├─ matematik.c
├─ giris.h
├─ giris.c
└─ main.c
```